

Прикладная теория типов

Домашнее задание 2 (просто типизированное λ -исчисление, алгебра типов)

14 октября 2025 г.

Домашняя работа принимается до 23:59 11 ноября 2025, кроме задач, помеченных звёздочкой, которые принимаются до конца семестра. Решения можно набрать в TeX или написать разборчивым текстом на бумаге и отсканировать. Домашняя работа принимается в виде **одного** PDF-файла (кроме задач, где нужно сдавать код или в тексте условия есть упоминание про отдельный PDF файл) на почту m.voronov@gse.cs.msu.ru. Вопросы по домашнему заданию можно задавать или по почте, или в ТГ-группе курса.

1. Найдите типы а-ля Карри и а-ля Чёрч для

- (2 балла) **zero** = $\lambda f x. x$
- (2 балла) **two** = $\lambda f x. f(fx)$
- (2 балла) **S** = $\lambda f g x. fx(gx)$

2. Определите, обитаемы ли данные типы в пустом контексте, если да, то приведите пример с соответствующим выводом а-ля Карри:

- (1 балл) $(\alpha \rightarrow \alpha \rightarrow \gamma) \rightarrow \alpha \rightarrow \beta \rightarrow \gamma$
- (1 балл) $((\alpha \rightarrow \gamma) \rightarrow \alpha) \rightarrow (\alpha \rightarrow \gamma) \rightarrow \beta \rightarrow \gamma$
- (2 балла)* $((\alpha \rightarrow \beta) \rightarrow \alpha) \rightarrow (\alpha \rightarrow \alpha \rightarrow \beta) \rightarrow \alpha$
- (2 балла)* $((\alpha \rightarrow \beta) \rightarrow \alpha) \rightarrow (\alpha \rightarrow \alpha \rightarrow \beta) \rightarrow \beta$

3. Найдите терм типа τ в контексте Γ с соответствующим выводом а-ля Карри:

- (1 балл) $\tau = (\alpha \rightarrow \beta) \rightarrow \alpha \rightarrow \gamma, \Gamma \equiv x : \alpha \rightarrow \beta \rightarrow \gamma$
- (1 балл) $\tau = \alpha \rightarrow (\alpha \rightarrow \beta) \rightarrow \gamma, \Gamma \equiv x : \alpha \rightarrow \beta \rightarrow \alpha \rightarrow \gamma$
- (1 балл) $\tau = (\alpha \rightarrow \gamma) \rightarrow (\beta \rightarrow \alpha) \rightarrow \gamma, \Gamma \equiv x : (\beta \rightarrow \gamma) \rightarrow \gamma$

4. Постройте замкнутые термы, удовлетворяющие следующим условиям:

- (2 балла) терм типа $(\alpha \rightarrow \beta) \rightarrow (\beta \rightarrow \alpha \rightarrow \gamma) \rightarrow \alpha \rightarrow \gamma$, которому нельзя приписать тип $\delta \rightarrow \delta \rightarrow \delta \rightarrow \gamma$.
- (3 балла) терм типа $(\alpha \rightarrow \beta) \rightarrow ((\alpha \rightarrow \beta) \rightarrow \gamma) \rightarrow \gamma$, которому нельзя приписать тип $\delta \rightarrow (\delta \rightarrow \gamma) \rightarrow \gamma$.

5. (3 балла) Для любого типа α определим тип $Option(\alpha)$, который удовлетворяет следующим правилам:

- в любом контексте справедливо следующее утверждение о типизации $None : Option(\alpha)$
- если $x : \alpha$ задан в контексте Γ , то справедливо $Some(x) : Option(\alpha)$
- если

- $e1 : \sigma$ в контексте Γ
- $e2 : \sigma$ в контексте $\Gamma, x : \alpha$
- $opt : Option(\alpha)$ в контексте Γ ,

то тогда выражение

match opt

$$\begin{aligned} None &\Rightarrow e1, \\ Some(x) &\Rightarrow e2 \end{aligned} \tag{1}$$

имеет тип σ в контексте Γ .

Запишите эти правила, как формальные правила вывода

- Выпишите β/η -законы для `match` и покажите, что `match` — это элиминация типа-суммы (см. слайды 9–11, где `match`=`case` и вычислительные правила)
6. Чему равна кардинальность следующих типов, в соответствии с принятыми на лекции обозначениями
- (1 балл) $\text{Either } \alpha (\text{Maybe } \beta)$
 - (1 балл) $(\text{Bool} \times \text{Bool}) \rightarrow \text{Either } \alpha \beta$
 - (1 балл) $\text{Either } (\text{Maybe } \alpha) (\text{Either } \beta (\text{Maybe } (\alpha \rightarrow \beta)))$
 - (2 балла) $\text{Void} \rightarrow \text{Bool}$, приведите также неалгебраическое обоснование, почему это так
 - (2 балла) $\text{Bool} \rightarrow \text{Void}$, приведите также неалгебраическое обоснование, почему это так
7. (4 балла) Докажите, что в заданном на лекции полукольце алгебраических типов, $|\alpha \rightarrow (\beta \rightarrow \gamma)| = |\alpha \times \beta \rightarrow \gamma|$, т.е. что кардинальность составной функции слева равна кардинальности функции из произведения типов справа. Приведите неалгебраическое обоснование, почему это так.
8. Какие распространённые структуры данных могут быть представлены следующими производящими функциями (для каждой задачи приведите обоснование):
- (2 балла) $F(\alpha) = 1 + \alpha \times \alpha \times F(\alpha)$
 - (5 баллов)

$$\begin{aligned} F(\alpha) &= 1 + \alpha \times G(\alpha) \\ G(\alpha) &= 1 + F(\alpha) \times G(\alpha) \end{aligned}$$

9. Закон экспонент (изоморфизм $\sigma^{\tau+\rho} \cong \sigma^\tau \times \sigma^\rho$).
- (2 балла) Постройте взаимно обратные функции:
 - `split` : $(\tau + \rho \rightarrow \sigma) \rightarrow (\tau \rightarrow \sigma) \times (\rho \rightarrow \sigma)$
 - `merge` : $(\tau \rightarrow \sigma) \times (\rho \rightarrow \sigma) \rightarrow (\tau + \rho \rightarrow \sigma)$
 - (1 балл) Докажите обратимость: `merge``split`  $\equiv_{\eta}$  `id` и `split``merge`  $\equiv_{\eta}$  `id` с использованием β/η -рассуждений.
 - (1 балл) Приведите неалгебраическое обоснование, почему эти типы изоморфны (например, на примере обработчиков запросов).

10. Зададим бинарное дерево

data *BinTree* α = *Leaf* | *Branch* α (*BinTree* α) (*BinTree* α),

докажите, что существует биективная функция

f : (*BinTree* α , *BinTree* α , *BinTree* α , *BinTree* α , *BinTree* α , *BinTree* α , *BinTree* α) $\rightarrow$ *BinTree* α ,

которая переводит кортеж из семи бинарных деревьев в одно бинарное дерево. Данная функция может использовать строение каждого дерева из домена только до глубины не больше 4 (в противном случае, существование такой функции доказать тривиально, потому что домен и кодомен функции - счётные множества). Это означает, что паттерн-матчинг любого дерева из домена при построении данной функции можно использовать только для глубины не более 4.

Данную задачу можно решить двумя способами:

- (7 баллов) непосредственно предъявив функцию, которая удовлетворяет этим условиям с обоснованием, почему она будет биективной. Функция может быть записана на любом распространённом языке программирования (удобнее использовать языки с паттерн-матчингом) или некотором понятном псевдо-коде. Указание: в этом варианте решения рассмотрите два случая, когда первые четыре дерева являются пустыми или нет.
- (3 балла) алгебраическим, доказав, что в полукольце $\frac{N}{x^2-x+1}$ минимальная степень $n > 2$, такая, что $x^n = x$, равна 7. Данное полукольцо является полукольцом, образованным алгебраическим представлением бинарного дерева с помощью производящей функции $F(\alpha) = 1 + F(\alpha)^2$.

11. Пусть $Tree\ a = \mu X. (1 + a \times X \times X)$. Определите $in : 1 + a \times Tree\ a \times Tree\ a \rightarrow Tree\ a$ и $unIn$ в обратную сторону, введите катаморфизм $cata$ на любом языке с паттерн-матчингом или псевдо-языке для $Tree\ a$ и реализуйте через него функции:

- (3 балла) **mirror** : $Tree\ a \rightarrow Tree\ a$ — зеркальное отражение дерева (левое и правое поддеревы меняются местами рекурсивно)
- (3 балла) **isBST** : $Tree\ Nat \rightarrow Bool$ — проверка свойства binary search tree (для каждого узла все элементы левого поддерева меньше значения узла, все элементы правого больше).

12. Опишите простыми словами для семилетнего ребёнка:

- (3 балла) **Теорема прогресса** для $\lambda_{\rightarrow} + \times + +$ (STLC с ADT). Объясните, что эта теорема гарантирует и почему это важно. Приведите 1–2 простые аналогии из повседневной жизни.
- (3 балла) **Изоморфизм типов** $A + A \cong Bool \times A$. Объясните на примерах, почему эти типы изоморфны. Можно использовать метафору с предметами из повседневной жизни.

13. **Fat pointers, vtables и экзистенциальные типы.** В этой задаче мы исследуем связь между trait objects (Rust) / виртуальными функциями (C++) и экзистенциальными типами из теории типов.

Цель: понять, как динамический полиморфизм реализуется на уровне типов и памяти, и как система типов гарантирует безопасность при type erasure. Это задание связывает абстрактные концепции (экзистенциальные типы) с конкретной реализацией в языках программирования.

Теоретическая часть

Краткая справка по экзистенциальным типам:

Экзистенциальный тип $\exists \alpha. T$ читается как «существует *некоторый* тип α , для которого выполняется T ». В отличие от универсального квантора $\forall \alpha. T$ (вызывающий код выбирает тип), при экзистенциальном кванторе конкретный тип α выбирается при создании значения, но *скрывается* от потребителя. Потребитель знает только, что «какой-то α существует», и может работать с ним только через предоставленный интерфейс.

Связь с trait objects: `Box<dyn Trait>` в Rust — это экзистенциальный тип: мы знаем, что внутри *какой-то* тип α , реализующий `Trait`, но не знаем какой именно. Аналогично в C++: указатель `Base*` на объект с виртуальными функциями скрывает конкретный производный тип (`Derived1`, `Derived2`, ...), оставляя доступными только методы базового класса.

- (2 балла) Trait object в Rust типа `dyn Trait` можно представить как экзистенциальный тип $\exists \alpha. (Ptr\ \alpha \times VTable_{Trait})$, где $VTable_{Trait}$ содержит указатели на реализации методов для конкретного (скрытого) типа α . Опишите концептуально:
 - Что означает «упаковка» (*pack*) значения типа T в trait object: как из `Box<T>` и реализации `impl Trait for T` получить `Box<dyn Trait>`?
 - Что означает «распаковка» (*unpack*): как через trait object можно вызвать метод, не зная конкретного типа T ?
 - Почему после упаковки информация о конкретном типе T становится недоступна? Как это связано с понятием *type erasure*?

Примечание: формальная запись правил введения/элиминации не требуется (но даёт дополнительные 2 балла за задачу) — достаточно концептуального объяснения с примерами на Rust/C++.

- (2 балла) Объясните, почему размер `Box<dyn Trait>` фиксирован (два указателя: указатель на данные + указатель на vtable), несмотря на то что под ним могут быть типы произвольного размера (`u8`, `[u8; 1024]`, структуры). Свяжите это с понятием *представления типа* (type representation) и косвенности (indirection).
- (2 балла) Нарисуйте схематично memory layout «толстого указателя» (fat pointer) для `Box<dyn Trait>` или `&dyn Trait`, явно показав два указателя (data ptr + vtable ptr), и сравните с layout обычного «тонкого указателя» (thin pointer) `Box<T>` или `&T`. Укажите размеры в байтах (для 64-битной архитектуры). Объясните, как это связано с тем, что `dyn Trait` — это DST (dynamically sized type), и почему компилятору нужна дополнительная метаинформация (vtable ptr) для работы с таким типом.

- (2 балла) Приведите пример trait'a в Rust (или интерфейса в C++), который **не может** быть trait object (не является *dyn-compatible*, ранее называлось *object-safe*). Объясните на уровне типов, почему система типов запрещает такое преобразование.

Практическая часть

- (6 баллов)* Реализуйте упрощённый механизм динамической диспетчеризации **вручную** без использования trait objects (Rust) или виртуальных функций (C++). Требования:
 - Определите trait/интерфейс с 2-3 методами (например, `trait Shape { fn area(&self) -> f64; fn perimeter(&self) -> f64; }`)
 - Создайте структуру `ShapeVTable`, содержащую указатели на функции, включая **обязательный деструктор/drop-функцию** для безопасного освобождения памяти (в Rust: `struct ShapeVTable { area: fn(*const ()) -> f64, perimeter: fn(*const ()) -> f64, drop: fn(*mut ()) }`). Деструктор критически важен для корректного управления памятью и предотвращения утечек — без него невозможно безопасно освободить объект неизвестного типа. Для мутабельных методов используйте `fn(*mut ())`. Можно добавить `size/align` для иллюстрации реальных метаданных.
 - Создайте структуру `FatPtr`, содержащую `data: *mut ()` (мутабельный для корректного drop) и `vtable: &'static ShapeVTable`
 - Реализуйте функции-конструкторы `fn new_circle(c: Circle) -> FatPtr`, `fn new_rectangle(r: Rectangle) -> FatPtr`, которые:
 - * Аллоцируют данные в куче (`Box::into_raw` в Rust или `new` в C++)
 - * Создают статическую `vtable` с указателями на конкретные реализации методов и drop-функцию (в Rust используйте `Box::from_raw` внутри drop)
 - * Возвращают `FatPtr`
 - Реализуйте методы вызова: `fn call_area(fp: &FatPtr) -> f64 { (fp.vtable.area)(fp.data) }`
 - Покажите использование: массив/вектор `Vec<FatPtr>`, где хранятся разные типы (`Circle`, `Rectangle`), и вызов методов через `vtable`
 - **Обязательно реализуйте `impl Drop for FatPtr`**, вызывающий `(self.vtable.drop)(self.data)` для корректного освобождения памяти и предотвращения утечек
- (2 балла)* В Rust используйте `*const ()` / `*mut ()`, `fn(*const ()) -> T` и `unsafe { *(ptr as *const T) }` для приведения типов. В C++ используйте `void*` и `static_cast`. Объясните, почему это требует **unsafe**/manual memory management и какие проверки компилятор не может выполнить (согласованность типа и `vtable`, корректное выравнивание, валидность указателей, соблюдение алиасинга).

Безопасность и инварианты

- (3 балла)* Опишите инварианты для корректности вашей реализации `vtable`. Например, `data` всегда указывает на валидный объект соответствующего типа. Какие undefined behaviors (use-after-free, type confusion, alignment issues, data races) могут возникнуть при нарушении этих инвариантов? Явно перечислите все **unsafe**-блоки и обоснуйте их корректность.
- (2 балла)* Объясните, как официальная реализация trait objects в Rust использует систему типов для гарантии этих инвариантов (lifetime checking, type safety при создании trait object). Сравните с C++: какие проблемы (object slicing при передаче по значению, dangling vtable pointers, забытый виртуальный деструктор) могут возникнуть и как их избежать?
- (1 балл)* Свяжите с концепцией «make illegal states unrepresentable»: почему `dyn Trait` может существовать только за указателем (`Box<dyn Trait>`, `&dyn Trait`, `Rc`, `Arc`, `Pin<...>`), но не как значение на стеке (`let x: dyn Trait = ...` запрещено)? Объясните связь с тем, что `dyn Trait` — это **dynamically sized type (DST)**: размер неизвестен в compile-time, поэтому тип может существовать только за указателем. Это гарантирует невозможность создать некорректное состояние на этапе компиляции.

(3 балла)* Реализуйте `downcast` — функцию, которая пытается преобразовать `FatPtr` обратно к конкретному типу. Предпочтительная сигнатура: `fn downcast<T>(fp: FatPtr) -> Result<Box<T>, FatPtr>`, чтобы при неудаче вернуть владение исходным объектом и избежать утечки памяти (при `Err(fp)` вызывающий может освободить через `Drop`). Для этого потребуется добавить в `vtable` поле `type_id: TypeId` (в Rust используйте `std::any::TypeId`, в C++ — `std::type_index`). **Использование `TypeId` явно разрешается и поощряется** для этой задачи. Важно понимать, что сравнение `TypeId` имеет сложность $O(1)$ (обычно это просто сравнение двух 64-битных значений или хешей), но даёт информацию **только о равенстве типов** — никакой рефлексии (introspection) полей, методов или имён типов через `TypeId` получить нельзя. Обсудите связь с рефлексией и `type erasure`: как хранение `TypeId` восстанавливает минимальную информацию о типе (идентичность), стёртую при создании `trait object`, не раскрывая структуру типа? Сравните с `Box<dyn Any>::downcast` из стандартной библиотеки.

Формат сдачи: Для практической части Git-репозиторий или zip-архив, содержащий:

- Исходный код реализации с подробными комментариями (особенно в `unsafe`-блоках)
- Рабочий пример использования (создание объектов разных типов, вызов методов, корректное освобождение памяти)

Для теоретической части используйте тот же формат сдачи, что и для остальных задач.

Рекомендуемые материалы:

- Rust Book. «[Using Trait Objects](#)» — основы `trait objects` и зачем нужен указатель
- Rust Nomicon. «[The Rustonomicon](#)» — главы про `trait objects`, `repr`, `DST`, `vtable`/метаданные
- Rust Reference. «[Traits — dyn-compatibility](#)» — полный список ограничений (обновлённая терминология)
- Huon Wilson. «[Peeking inside Trait Objects](#)» — детальный разбор внутреннего устройства `trait objects`
- Arthur O'Dwyer. «[What is Type Erasure?](#)» — про `type erasure` в C++
- Laurence Tratt. «[A Quick Look at Trait Objects in Rust](#)» — объяснение `fat pointers`

14. (6 баллов)* **SKI-квесты в онлайн-песочнице.**

В данном задании можно решать любые задачи с страницы квестов [Simple Kombinator Interpreter — Quests](#). За каждые 2 решённые задачи начисляется 1 балл, максимум — 6 баллов.

Формат сдачи: Отдельный PDF-документ со списком решённых квестов (укажите номера/названия), основными частями редукций и краткими комментариями.

15. (15-30 баллов)* Выберите open-source проект с числом звёзд больше 100 на GitHub (или другой платформе), в котором активно используются алгебраические типы данных (ADT) или обобщённые алгебраические типы данных (GADT). Проведите анализ проекта по аналогии с разбором `quinn-proto` из Лекции 5, включая следующие аспекты:

- Описание основных алгебраических типов данных в проекте (произведения, суммы, рекурсивные типы)
- Анализ использования паттерн-матчинга и `exhaustiveness checking`
- Применение `phantom types`, `newtypes` или `smart constructors` (если есть)
- Использование классов типов/`traits` для абстракции (если применимо)
- Примеры из кодовой базы, демонстрирующие принцип «`make illegal states unrepresentable`»
- Связь дизайна типов с безопасностью и корректностью (инварианты, гарантии на уровне типов)
- Критический анализ: что сделано хорошо, что можно улучшить с точки зрения типового дизайна

Требования к проекту:

- Число звёзд на GitHub ≥ 100 (или эквивалент на другой платформе)
- Вы не являетесь автором или мажорным контрибьютором проекта
- Проект может быть написан на любом языке программирования (Rust, Haskell, OCaml, F*, Scala, TypeScript, и т.д.)

- В проекте должны активно использоваться ADT/GADT (не просто структуры данных, а именно алгебраические типы с вариантами)

Примеры подходящих проектов (можно выбрать любой другой, удовлетворяющий требованиям):

Сетевые протоколы и криптография:

- [rustls](#) — TLS библиотека на Rust
- [ocaml-tls](#) — Pure OCaml TLS
- [Sequoia-PGP](#) — OpenPGP в Rust

Парсеры и компиляторы:

- [tree-sitter](#) — инкрементальный парсинг
- [nom](#) — Parser combinators в Rust
- [rustc frontend](#) — HIR/MIR типы компилятора Rust

Базы данных и storage:

- [sled](#) — Embedded database в Rust
- [TiKV](#) — Distributed KV с Raft state machine

Распределённые системы:

- [Tendermint](#) — Byzantine consensus
- [OpenEthereum](#) — Ethereum client

Другие домены:

- [Servo](#) — Browser engine (CSS, DOM типы)
- [Bevy](#) — Game engine (ECS через типы)

Формат сдачи: отдельный PDF-документ с примерами кода, диаграммами типов и анализом. Можно включить собственные упрощённые модели типов из проекта (по аналогии с упрощённой моделью QUIC из Лекции 4).

Критерии оценки:

- **15-19 баллов** (базовый уровень): описание основных типов в проекте (или части проекта), примеры паттерн-матчинга, поверхностный анализ. Минимальная связь с концепциями из лекций.
- **20-24 балла** (хороший уровень): детальное описание ADT с примерами из кода, анализ exhaustiveness checking, примеры применения phantom types/newtypes, связь с безопасностью. Явная связь с концепциями F/I/E/C, алгебра типов.
- **25-27 баллов** (отличный уровень): всё из предыдущего уровня + глубокий критический анализ дизайна, конкретные предложения по улучшению, нетривиальные наблюдения об использовании типов, собственная упрощённая модель части системы.
- **28-30 баллов** (максимальный уровень): всё из предыдущего + формальный анализ инвариантов, связь с теоремами о типах, сравнение с альтернативными подходами.